SEMINAR REPORT
ON

Loan Default Risk Prediction using Random Forest

BY

Name : Prasad Bhalerao **RBT NO: RBT24CB026**

Third Year B. TECH
ACADEMIC YEAR: 2026-27

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS

JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING TATHAWADE,
PUNE - 411033

CERTIFICATE

This is to certify that the seminar report entitled **"Loan Default Risk Prediction using Random Forest"** submitted by Prasad Bhalerao (PRN No. RBT24CB026) is a record of bonafide work carried out under my supervision, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Business Systems of Savitribai Phule Pune University, during the academic year 2026-27.

Date:

Prof. Neeraj Sathwale	**Dr. K. P. Moholkar**	**Dr. A. M. Badadhe**
Seminar Guide	H. O. D. CSBS	Director, R.S.C.O.E. Pune

Acknowledgement

I express my deep sense of gratitude to my seminar guide, Prof. Neeraj Sathwale, Department of Computer Science and Business Systems, for the valuable guidance, continuous encouragement and constructive feedback given throughout the course of this seminar work.

I would like to extend my sincere thanks to Dr. K. P. Moholkar, Head of the Department of Computer Science and Business Systems, for providing the facilities and academic support necessary to complete this work.

I also express my heartiest gratitude to Dr. A. M. Badadhe, Director, for the constant support and encouragement.

Finally, I am grateful to all the faculty members and staff of the Department of Computer Science and Business Systems, and to my family and friends, for their help and motivation during the preparation of this report.

Prasad Bhalerao
RBT24CB026
Department of Computer Science and Business Systems
JSPM's Rajarshi Shahu College of Engineering, Pune

Abstract

With the rapid advancement of machine learning in financial services, credit risk assessment has emerged as a crucial area of focus due to its ability to support lending decisions from historical borrower data rather than judgement alone. This seminar studies the application of the Random Forest algorithm to the prediction of serious borrower delinquency, and reports a complete, working implementation. The study uses the public Kaggle "Give Me Some Credit" dataset of 150,000 labelled borrower records described by 10 numeric attributes. The target variable, SeriousDlqin2yrs, records whether a borrower experienced delinquency of 90 days or worse within two years of the observation date, and only 6.68% of records are positive. The work establishes that accuracy is an unsuitable headline metric for such an imbalanced problem, because a model that always predicts the majority class attains 93.3% accuracy while identifying no delinquent borrower at all. A preprocessing pipeline was constructed to treat undocumented placeholder codes and missing values, five Random Forest configurations were compared on a held-out validation split using the area under the precision-recall curve, and the decision threshold was selected on validation data rather than assumed to be 0.50. On a test split of 30,000 records, scored exactly once, the final model achieved a ROC-AUC of 0.8654 and a PR-AUC of 0.4014 against a no-skill baseline of 0.0668. The trained pipeline was deployed as a REST service with a web interface, demonstrating the complete path from raw data to a usable application. The report also documents the limitations that prevent such a system from being used for real credit decisions.

Keywords: Random Forest, Credit Risk Prediction, Class Imbalance, Precision-Recall Curve, Decision Threshold, Supervised Classification

INDEX

Sr. No.	Contents	Page No.
1	Acknowledgement	iii
2	Abstract	iv
3	List of Figures	vi
4	List of Tables	vi
5	Chapter 1: Introduction	1
6	Chapter 2: Literature Review	3
7	Chapter 3: Technology / Concept Overview	6
8	Chapter 4: Applications and Use Cases	12
9	Chapter 5: Advantages and Limitations	21
10	Chapter 6: Future Scope	23
11	Chapter 7: Conclusion	24
12	References / Bibliography	26

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
Fig. 3.1	System architecture of the implemented application	7
Fig. 3.2	Training pipeline from raw data to saved model artifact	8
Fig. 3.3	Stratified 60/20/20 split and the role of each part	8
Fig. 3.4	How a Random Forest produces a prediction	9
Fig. 4.1	Dashboard page of the deployed application	14
Fig. 4.2	Borrower input form on the Predict page	15
Fig. 4.3	Prediction result for a higher-risk borrower	16
Fig. 4.4	Model information page with evaluation metrics	17
Fig. 4.5	Predicted score distribution by actual outcome	19
Fig. 4.6	Precision, recall and F1 across decision thresholds	19

LIST OF TABLES

Table No.	Title of Table	Page No.
Table 2.1	Summary of literature review	4
Table 3.1	Key concepts and definitions	7
Table 3.2	Input features and their descriptions	9
Table 3.3	Data quality issues and their treatment	10
Table 3.4	Validation comparison of model configurations	11
Table 4.1	Held-out test set evaluation metrics	18
Table 4.2	Confusion matrix on the test set	18
Table 4.3	Per-class classification report	18
Table 4.4	Effect of the decision threshold	19
Table 5.1	Advantages and limitations compared	22
Table 7.1	Technology stack	24

CHAPTER 1: INTRODUCTION

1.1 Overview of the Topic

Credit risk prediction is the task of estimating, from information known about a borrower, how likely that borrower is to fail to meet their repayment obligations. Traditionally this judgement was made by human underwriters applying policy rules to an application form. Machine learning changes the basis of the judgement: instead of encoding rules by hand, a model is fitted to a historical record of borrowers whose outcomes are already known, and it learns the statistical relationship between borrower attributes and those outcomes.

This seminar studies that application through one specific algorithm, the Random Forest, and one specific prediction target. A Random Forest is an ensemble method: it grows many decision trees on randomised samples of the training data and combines their outputs. It is widely used for tabular problems of this kind because it captures non-linear relationships and interactions between variables without manual feature engineering, is robust to extreme values, and produces probability estimates rather than only hard labels.

The problem studied here is formally a supervised binary classification task. Each record describes one borrower through 10 numeric attributes, and the label states whether that borrower experienced serious delinquency — defined as 90 or more days past due — within two years of an observation date. It is not a regression problem, because the quantity predicted is a category rather than a magnitude, and it is not a clustering problem, because the groups are given in advance by the labels rather than discovered in the data.

1.2 Need and Importance

Lending institutions extend credit under uncertainty. A borrower who defaults imposes a direct loss, while a sound borrower who is wrongly refused represents forgone revenue and a denied customer. Any method that improves the ranking of applicants by risk therefore has direct economic value, and the scale of modern lending makes manual assessment of every application impractical.

The problem is also statistically interesting in a way that makes it a good subject for study. Serious delinquency is rare: in the dataset used here only 6.68% of borrowers

are positive. This imbalance invalidates the most familiar evaluation metric. A model that ignores its inputs entirely and always answers “no delinquency” achieves 93.3% accuracy while identifying not a single delinquent borrower. Reporting accuracy for such a problem is therefore misleading, and a central theme of this seminar is which metrics should be used instead and why.

Understanding these issues matters beyond credit scoring. Rare-event prediction under class imbalance appears in fraud detection, medical diagnosis, equipment failure prediction and network intrusion detection. The evaluation reasoning developed in this report transfers directly to all of them.

1.3 Objectives of the Seminar

The main objectives of this seminar are:

- To understand the fundamental concepts of the Random Forest algorithm, including bootstrap aggregation, random feature selection at each split, and how an ensemble of trees produces a probability estimate.
- To study the architecture and working of a complete prediction system, from raw data through preprocessing and model selection to a deployed service.
- To examine the correct evaluation of a class-imbalanced classifier, and to demonstrate why accuracy is inadequate and what should replace it.
- To implement the method on a real public dataset and report honestly measured results rather than illustrative ones.
- To explore real-world applications of credit risk prediction and the benefits it offers to industry.
- To identify the advantages, limitations and ethical challenges of such systems, and to examine future developments in the field.

CHAPTER 2: LITERATURE REVIEW

This chapter reviews the principal sources consulted in preparing this seminar, covering the algorithm itself, the evaluation of imbalanced classifiers, and the software used for the implementation.

Breiman [1] introduced Random Forests and established the theoretical basis for the method. The paper shows that combining many decorrelated trees reduces the variance of the prediction without a corresponding increase in bias, and that the generalisation error converges as more trees are added rather than diverging through overfitting. Two sources of randomness are responsible: bootstrap sampling of the training rows, and the restriction of each split to a random subset of features. The second is what decorrelates the trees, and it is the mechanism that makes the averaging effective.

Saito and Rehmsmeier [4] address the evaluation problem directly and were the most influential source for this work. They demonstrate that the ROC plot is misleading when the classes are imbalanced, because the false-positive rate is computed against the large negative population and therefore moves very little even when the number of false alarms is substantial. The precision-recall plot does not suffer from this, because precision is computed only against the cases the model actually flagged. This finding is the reason the present work selects its model on PR-AUC rather than ROC-AUC or accuracy.

Pedregosa et al. [2] document the scikit-learn library used for the implementation. The documentation of RandomForestClassifier was consulted directly on one point of substance: the library computes a predicted probability as the mean of the per-tree class probabilities, where each tree contributes the class proportion of the leaf a sample reaches. This is not the hard majority vote that introductory treatments often describe, and the distinction is preserved throughout this report.

Chawla et al. [5] survey the broader family of approaches to class imbalance, including resampling methods such as SMOTE and cost-sensitive learning through class weights. Their framing of the problem as one of choosing an operating point on a trade-off curve, rather than of fixing a defective model, informed the decision in this work to adjust the decision threshold rather than reweight the classes.

The dataset itself originates from the Kaggle "Give Me Some Credit" competition [3], which supplied the labelled borrower records and the data dictionary defining each attribute.

Table 2.1: Summary of literature review

Sr. No.	Author(s) and Year	Research Focus	Key Findings
1	Breiman (2001)	Random Forest algorithm and its theory	Averaging decorrelated trees reduces variance; generalisation error converges as trees are added. Bootstrap sampling and random feature subsets are the two sources of randomness.
2	Pedregosa et al. (2011)	scikit-learn machine learning library	Provides the reference implementation used here. Predicted probability is the mean of per-tree leaf class proportions, not a majority vote.
3	Kaggle / Credit Fusion (2011)	Give Me Some Credit competition dataset	Supplies 150,000 labelled borrower records with a documented data dictionary; competition was scored on AUC.
4	Saito and Rehmsmeier (2015)	Evaluation of imbalanced binary classifiers	ROC plots are over-optimistic under imbalance; precision-recall plots are more informative because precision ignores the large true-negative population.
5	Chawla et al. (2002, 2004)	Class imbalance: resampling and cost-sensitive learning	Frames imbalance as a choice of operating point on a trade-off curve; resampling and class weighting shift that point rather than improving the underlying ranking.

Major observations and research gaps.

Three observations emerge from the reviewed sources. First, the Random Forest is well understood theoretically and is a sound default for tabular classification, so the interesting difficulty in a project of this kind lies not in the algorithm but in the data and the evaluation. Second, the literature is consistent that accuracy and ROC-AUC flatter a classifier when the positive class is rare, yet both remain widely reported in practice. Third, the standard remedies for imbalance are frequently applied without checking whether they improve the model's ranking or merely relocate its operating point.

The gap addressed by the practical component of this seminar is that last point. Rather than applying class weighting because it is conventional, this work measures its effect and finds that it moves the model along the precision-recall curve without lifting it,

and therefore uses explicit threshold selection instead. A second gap concerns undocumented placeholder values in the dataset, which much published work on this data removes or imputes silently; here they are shown to carry substantial signal and are handled explicitly.

CHAPTER 3: TECHNOLOGY / CONCEPT OVERVIEW

3.1 Basic Concepts

This section defines the terminology used throughout the remainder of the report.

Supervised classification. A model is fitted to records whose correct labels are known, and learns a mapping from attributes to a category. Here the category is binary.

Decision tree. A predictor that repeatedly splits the data on one attribute at a time. Each split is chosen to separate the classes as well as possible; the terminal nodes, called leaves, hold the training samples that reached them.

Gini impurity. The measure of how mixed a node is, defined as $G = 1 - \Sigma\, pk^2$, where pk is the proportion of class k in that node. A node containing a single class has impurity zero. Each split is chosen to maximise the weighted reduction in G.

Bootstrap aggregation (bagging). Training each tree on a sample drawn with replacement from the training set, so that every tree sees a slightly different dataset. Averaging their predictions reduces variance.

Class imbalance. The condition in which one class is far rarer than the other. Here the positive class is 6.68% of records.

Decision threshold. The probability above which a prediction is reported as positive. It is a choice made after training, and it determines where the model sits on the trade-off between precision and recall.

Table 3.1: Key concepts and definitions

Term	Definition
Random Forest	An ensemble of 300 decision trees grown on bootstrap samples, with each split restricted to a random subset of features.
Target variable	SeriousDlqin2yrs: 1 if the borrower was 90+ days past due within two years, else 0.
Precision	Of the borrowers flagged as delinquent, the proportion who truly were. TP / (TP + FP).
Recall	Of the borrowers who truly were delinquent, the proportion the model identified. TP / (TP + FN).
F1-score	The harmonic mean of precision and recall, 2PR / (P + R), which is dominated by whichever is weaker.
ROC-AUC	The probability that a randomly chosen positive is scored above a randomly chosen negative. 0.5 is chance.
PR-AUC	Area under the precision-recall curve. Its no-skill baseline equals the positive rate, here 0.0668.

3.2 Working Principle / Architecture

The implemented system separates offline work from online serving. Training, evaluation and figure generation are performed offline and produce a single saved artifact. The web service loads that artifact once when it starts and uses it to answer requests; it never trains.

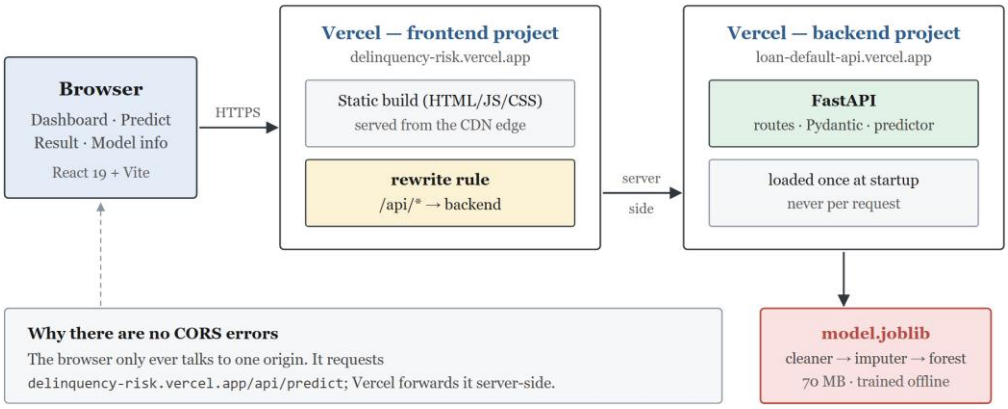


Figure 3.1: System architecture of the implemented application

The browser communicates with one origin only. Requests to the /api path are forwarded to the prediction service on the server side, so no cross-origin request is made by the browser at any point. The model file is bundled with the service and loaded a single time at startup rather than on each request, which would otherwise dominate the response time.

Training pipeline.

Training proceeds through six stages. The labelled file is loaded and validated against an expected schema; implausible values are marked as missing; the data is split; candidate models are compared; the decision threshold is selected; and finally the chosen model is scored once on data it has never seen.

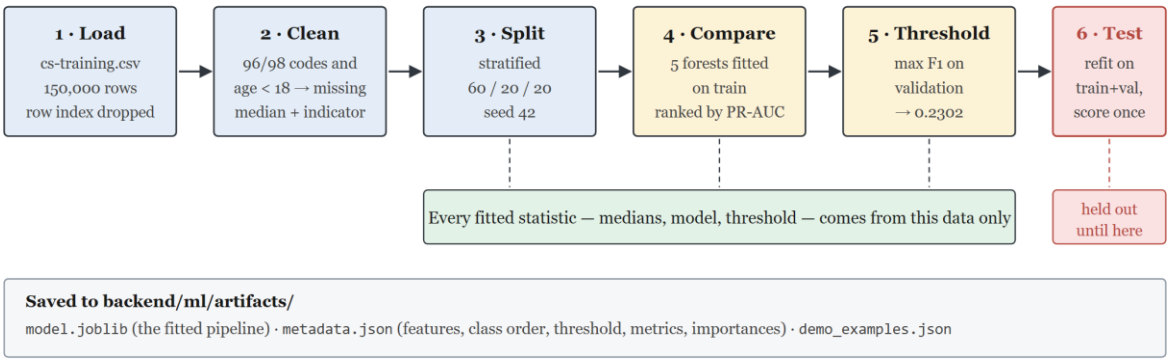


Figure 3.2: Training pipeline from raw data to the saved model artifact

The discipline that makes the final figures trustworthy is that every fitted quantity — the imputation medians, the model parameters and the decision threshold — is derived from the training and validation data only. The test split is held in reserve until the last stage.

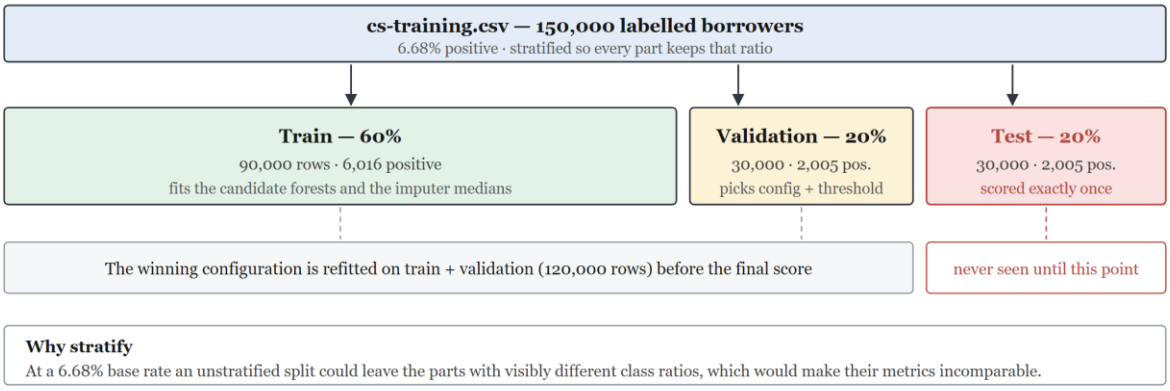


Figure 3.3: Stratified 60/20/20 split and the role of each part

The split is stratified, meaning each part preserves the overall class ratio. At a 6.68% positive rate an unstratified split could leave the three parts with noticeably different base rates, which would make their metrics incomparable. Figure 3.3 confirms that all three parts carry the same 6.68% positive rate.

How a prediction is produced.

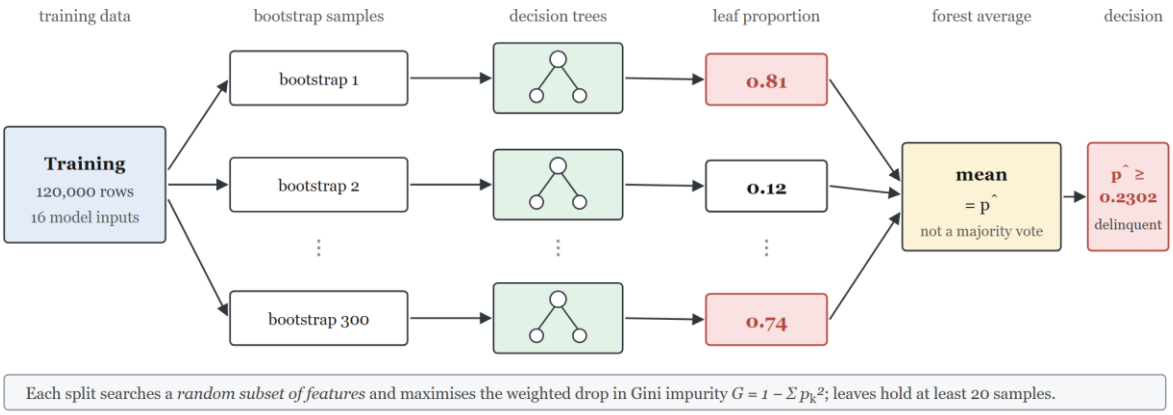


Figure 3.4: How a Random Forest produces a prediction

For a new borrower, each of the 300 trees routes the record down to one leaf and reports the proportion of positive training samples in that leaf. The forest averages those 300 proportions, and the mean is the predicted probability. This is an important detail of the implementation: scikit-learn averages per-tree probabilities rather than taking a hard majority vote of per-tree classes. The resulting probability is then compared with the decision threshold of 0.2302 to produce the reported class.

3.3 Key Features

The dataset provides ten numeric attributes per borrower, described in Table 3.1 below. No categorical encoding is required, and because decision trees split on rank order rather than magnitude, no feature scaling is required either.

Table 3.2: Input features and their descriptions

No.	Feature	Description
1	RevolvingUtilizationOfUnsecuredLines	Card and personal line balances divided by total credit limits
2	age	Borrower age in years
3	NumberOfTime30-59DaysPastDueNotWorse	Count of 30–59 day late episodes in the last two years
4	DebtRatio	Debt payments, alimony and living costs divided by gross monthly income
5	MonthlyIncome	Gross monthly income (19.82% missing)
6	NumberOfOpenCreditLinesAndLoans	Open installment loans and lines of credit
7	NumberOfTimes90DaysLate	Count of 90+ day late episodes
8	NumberRealEstateLoansOrLines	Mortgages and home equity lines of credit
9	NumberOfTime60-89DaysPastDueNotWorse	Count of 60–89 day late episodes in the last two years
10	NumberOfDependents	Dependents excluding the borrower (2.62% missing)

3.4 Components and Their Interaction

Preprocessing and the classifier are combined into a single scikit-learn Pipeline, and it is that composite object which is saved to disk. The serving code loads the pipeline and calls it, so there is no second copy of the cleaning or imputation logic on the serving side. This makes a mismatch between training and inference structurally impossible rather than merely something to be careful about.

The pipeline has three stages in sequence. A cleaning step replaces documented placeholder and impossible values with missing markers. An imputation step fills those gaps with the training median and additionally appends a binary indicator column recording that the value was absent. The Random Forest classifier is the final stage. A consequence worth noting is that the classifier receives 16 columns built from the 10 input features, because of the appended indicators; the public interface of the service is unchanged.

Table 3.3: Data quality issues and their treatment

Issue found in the raw data	Extent	Treatment applied
MonthlyIncome missing	29,731 rows (19.82%)	Median imputation plus a missing-value indicator
NumberOfDependents missing	3,924 rows (2.62%)	Median imputation plus a missing-value indicator
Late-payment counters coded 96 or 98	269 rows	Treated as placeholders; imputed with an indicator, because these rows are 54.65% positive against a 6.68% base rate
Age recorded as 0	1 row	Impossible for a credit applicant; treated as missing
Duplicate predictor rows	646 rows	Retained; plausibly distinct borrowers with the same rounded values
Extreme ratio values	Up to 329,664	Retained; a Random Forest splits on rank order and is unaffected

Model selection.

Five configurations were fitted on the training split and compared on the validation split. The selection metric was PR-AUC, chosen for the reasons given in the literature review.

Table 3.4: Validation comparison of model configurations

Configuration	ROC-AUC	PR-AUC	Precision	Recall
baseline	0.8450	0.3481	0.5334	0.1870
regularised	0.8615	0.3820	0.5988	0.1556
balanced	0.8602	0.3776	0.2399	0.7197
balanced subsample	0.8602	0.3780	0.2805	0.6504
regularised with indicators	0.8630	0.3863	0.6290	0.1581

Two findings from this comparison shaped the final model. Increasing the minimum leaf size from the default to 20 improved validation PR-AUC from 0.3481 to 0.3820, confirming that an unrestricted forest was overfitting. Class weighting, by contrast, raised recall from 0.1556 to 0.7197 but left PR-AUC essentially unchanged at 0.3776, which means it moved the model along the precision-recall curve without lifting the curve itself. Since a decision threshold achieves the same movement while remaining visible and adjustable without refitting, class weighting was not used.

The winning configuration added missing-value indicator columns, which gave the best validation PR-AUC of 0.3863. The decision threshold was then selected on the validation split by maximising F1, giving 0.2302 rather than the conventional 0.50. The model was refitted on train and validation combined and scored once on the test split.

CHAPTER 4: APPLICATIONS AND USE CASES

4.1 Applications

Credit risk models of this kind are applied across the financial sector and beyond. In retail lending they support decisions on personal loans, credit cards and mortgages. In banking operations they drive portfolio monitoring, identifying accounts whose risk profile has deteriorated since origination. In regulatory reporting they feed the expected-loss calculations that determine how much capital an institution must hold.

The same technique generalises to any rare-event prediction problem on tabular data. Fraud detection, insurance claim triage, customer churn prediction, predictive maintenance of equipment and clinical risk scoring all share the structure studied here: many records, a minority class of interest, and a need for a ranked score rather than a bare label.

4.2 Real-World Use Cases

Application scoring at origination. When an application is received, the lender must decide whether to extend credit. A model trained on past applicants produces a risk score that is combined with policy rules and affordability checks. The benefit is consistency and speed: every applicant is assessed on the same basis in milliseconds. The present system demonstrates this use case directly.

Behavioural scoring of an existing portfolio. For borrowers already on the books, payment behaviour accumulates over time. Rescoring the portfolio periodically identifies accounts drifting toward delinquency early enough for intervention, such as a payment plan. The payment-history attributes that dominate this model are precisely the ones that update as behaviour changes.

Collections prioritisation. Where a portfolio already contains delinquent accounts and recovery resources are finite, a ranked score directs effort to the accounts where it will matter most. This use case needs only the ranking, not a calibrated probability, which makes it well suited to a model with a strong ROC-AUC.

4.3 Benefits to Industry and Business

The principal benefits are consistency, scale and transparency of reasoning. An automated score applies identical criteria to every applicant, removing the variation

between individual assessors. It scales to volumes no manual process could handle. And because a Random Forest exposes which attributes it relies on, the resulting model can be inspected and discussed rather than treated as opaque.

For the institution the economic effect is a better-informed trade-off between the loss from a defaulted loan and the margin forgone on a declined application. The threshold analysis in Section 4.5 shows exactly how that trade-off can be tuned to an institution's cost structure.

4.4 Implemented Application Interface

The trained model was deployed as a web application to demonstrate the complete path from data to a usable system. It is publicly available at https://delinquency-risk.vercel.app, with the service's own interactive API documentation at https://loan-default-api.vercel.app/docs. The interface has four pages, shown in the figures that follow. All values displayed are fetched from the prediction service at run time; none are fixed in the page.

Dashboard.

The landing page states what the model predicts and what it does not, and reports the measured test metrics. It deliberately presents the majority-class baseline alongside the accuracy figure, so that a reader cannot mistake the accuracy for a strong result.

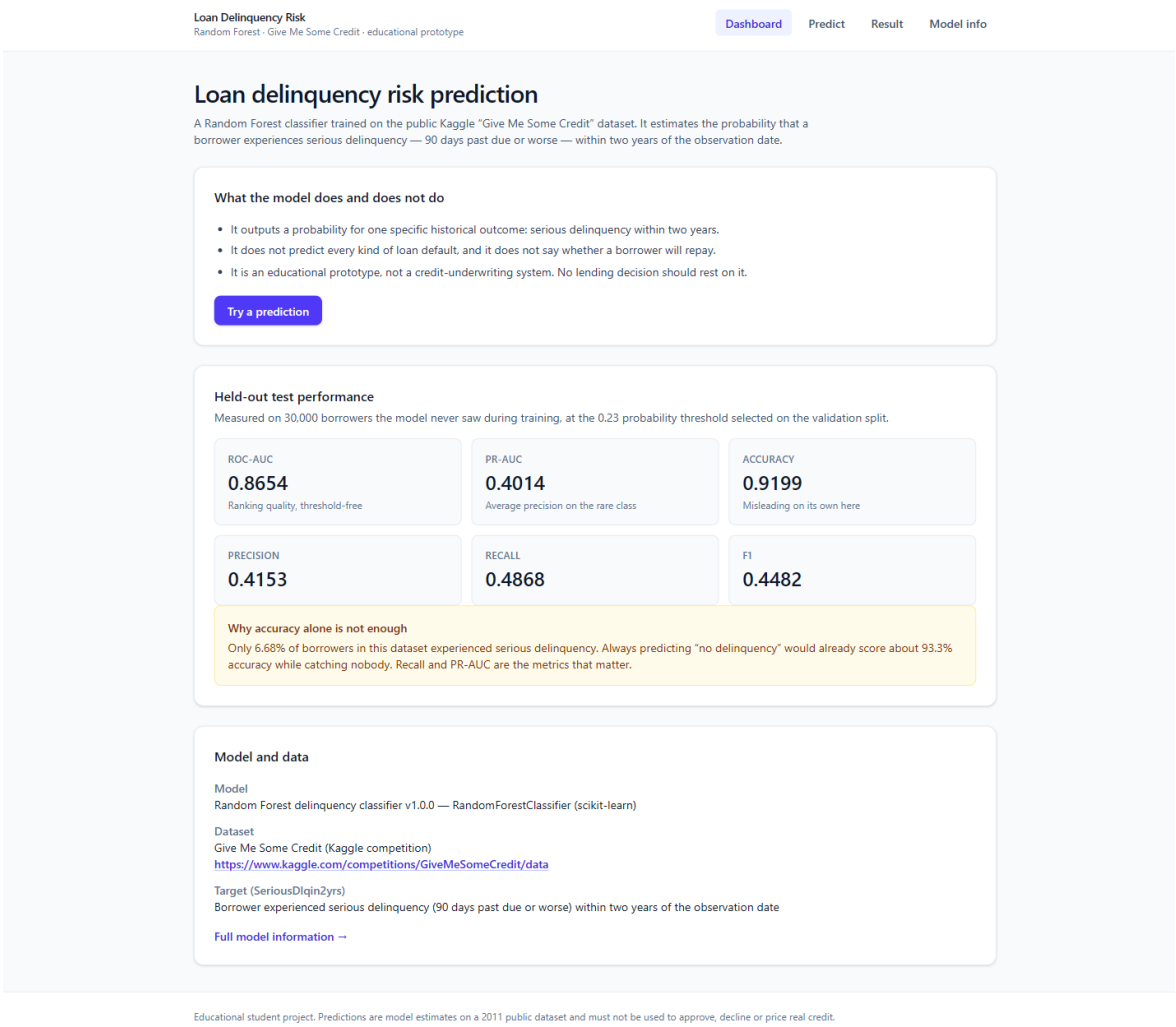


Figure 4.1: Dashboard page of the deployed application

Prediction form.

The form exposes exactly the ten attributes the model was trained on, grouped into borrower profile, credit usage and payment history. Technical column names are replaced with readable labels, less obvious financial variables carry short explanations, and the two optional fields are marked as such. A demonstration button loads a genuine record from the held-out test split, labelled as such so it is not mistaken for information about the person using the page.

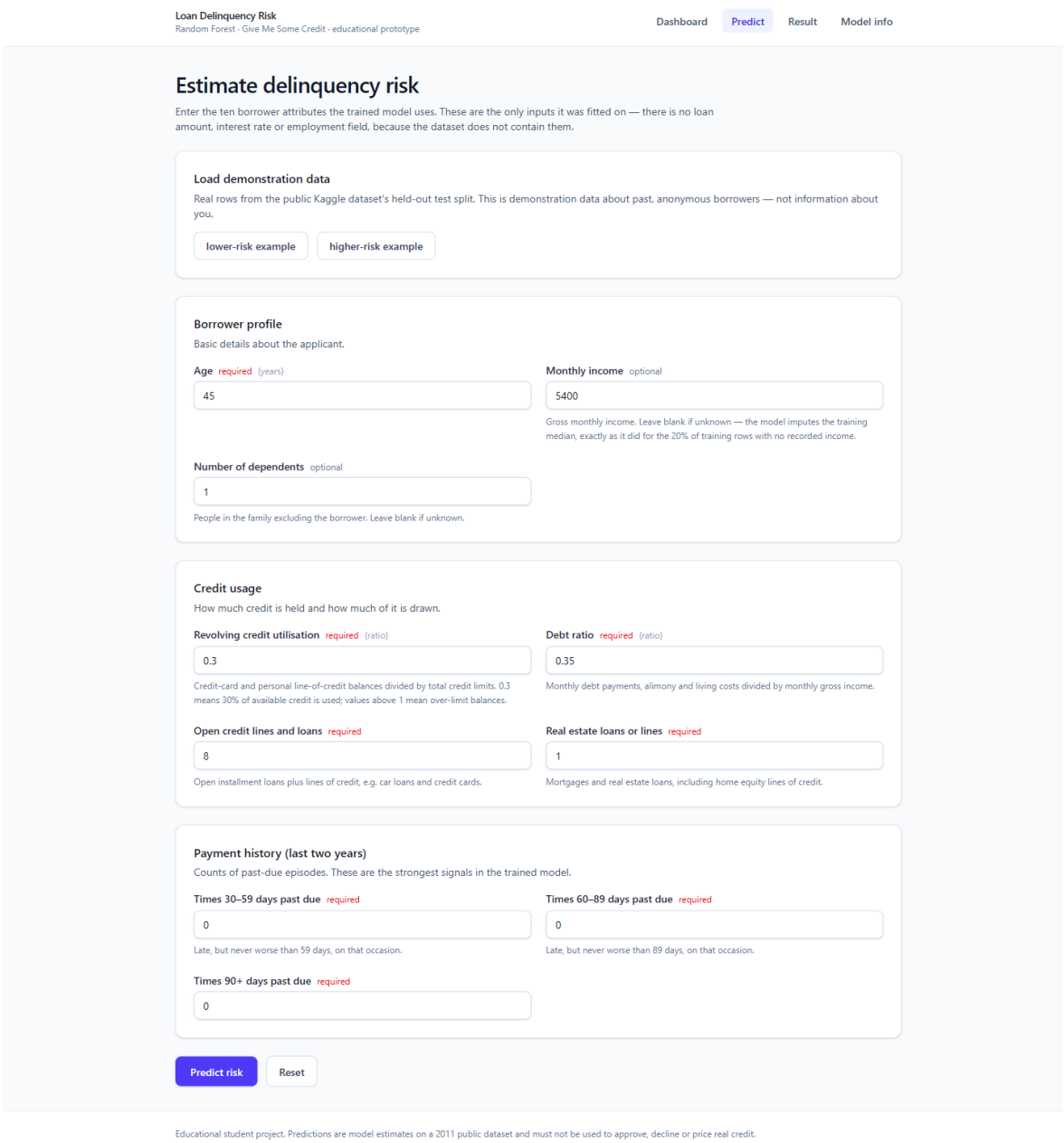


Figure 4.2: Borrower input form on the Predict page

Prediction result.

The result page separates three things that are easily conflated: the estimated probability, the binary class derived from the threshold, and an illustrative risk band. The lower panel places the individual score against the distribution of scores for borrowers whose actual outcome is known, so that a single number is given a frame of reference.

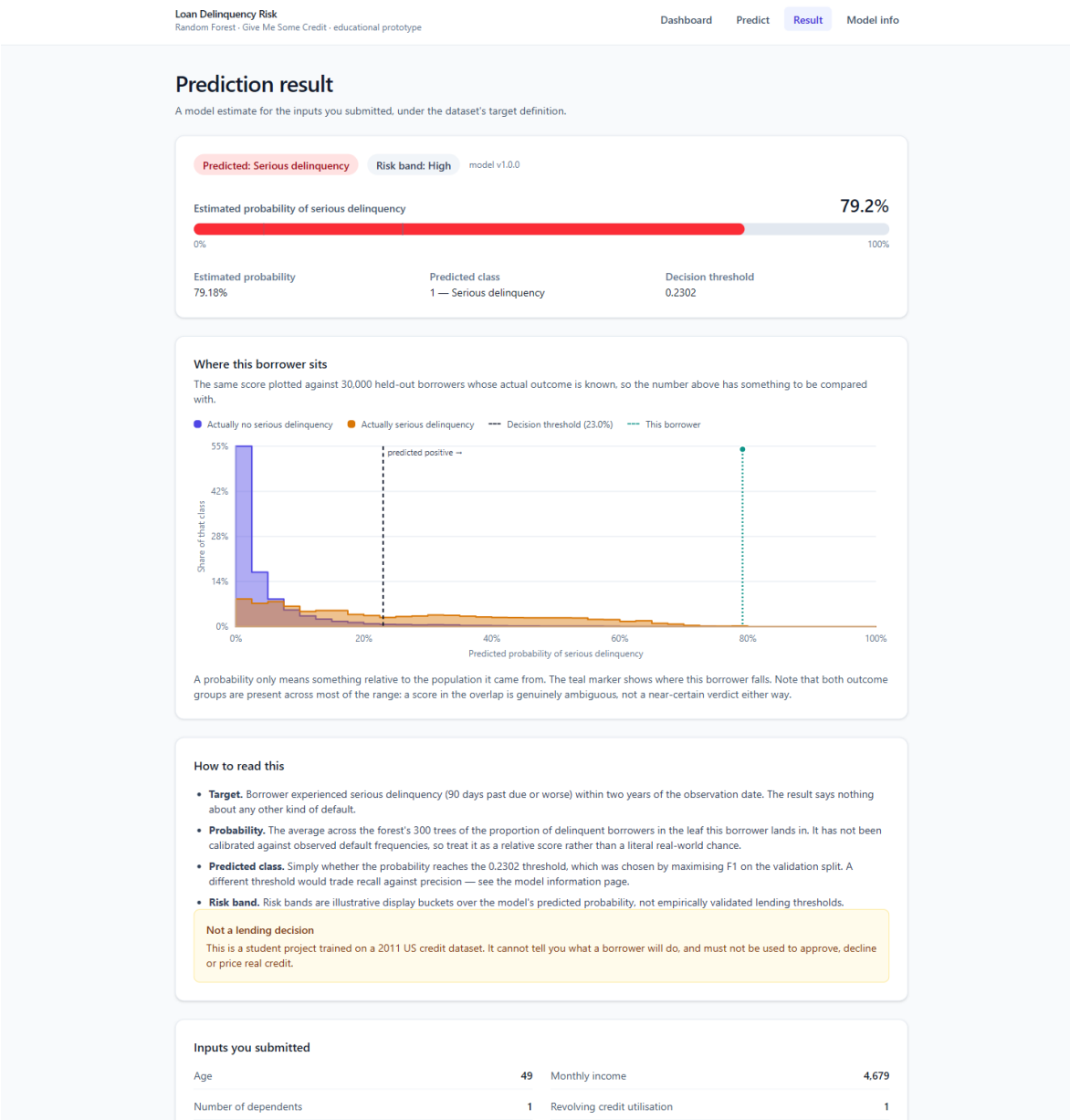


Figure 4.3: Prediction result for a higher-risk borrower

Model information.

The final page reports the algorithm, the dataset and its source, the target definition, the decision threshold and how it was chosen, the full evaluation metrics and the feature importances.

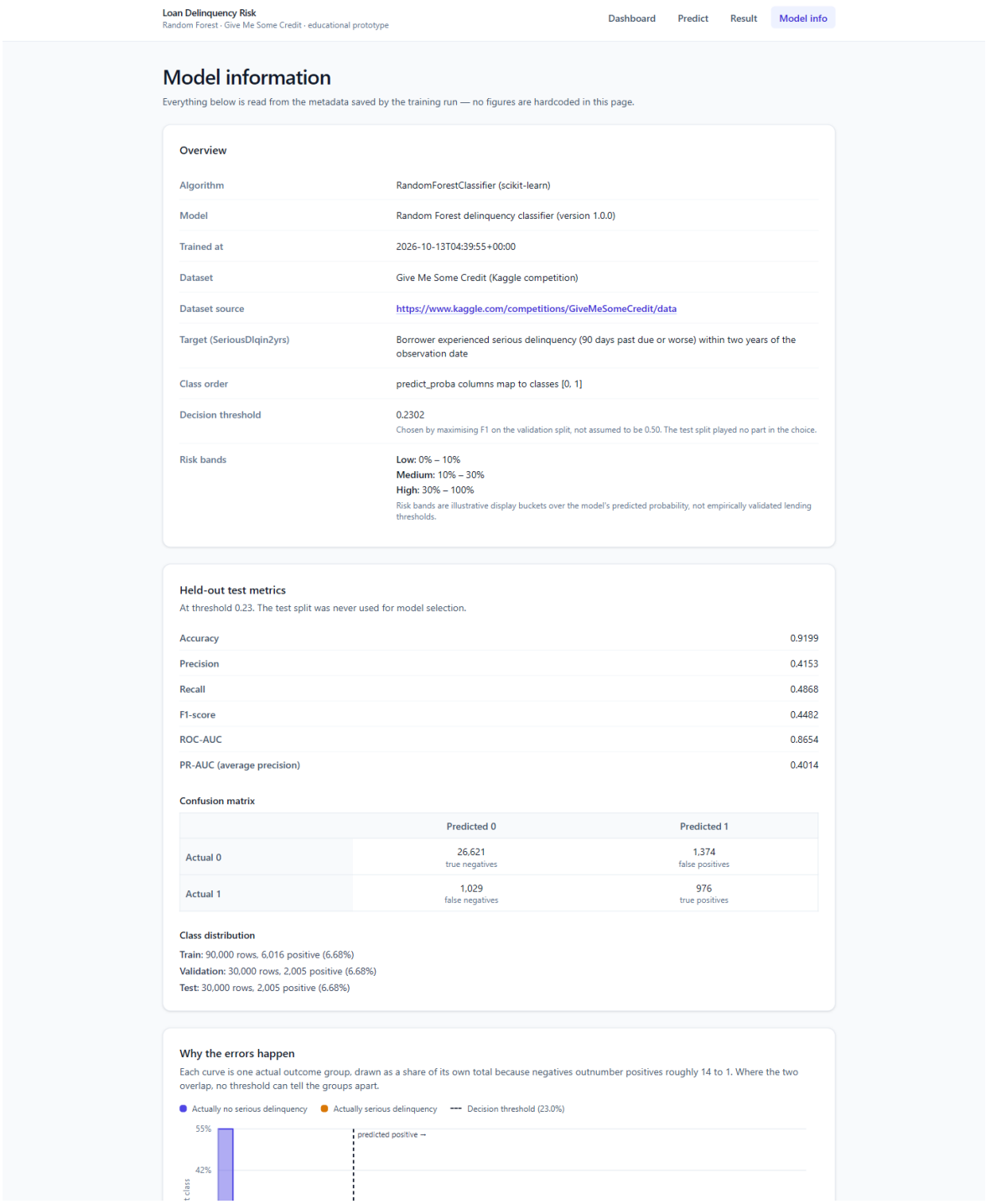


Figure 4.4: Model information page with evaluation metrics

4.5 Results Obtained

The figures below were produced by scoring the held-out test split of 30,000 borrowers exactly once, at the validation-selected threshold of 0.2302.

Table 4.1: Held-out test set evaluation metrics

Metric	Value	Interpretation
ROC-AUC	0.8654	Ranks a delinquent borrower above a sound one 86.5% of the time
PR-AUC	0.4014	About six times the no-skill baseline of 0.0668
Recall	0.4868	Identified 976 of 2,005 real delinquencies
Precision	0.4153	Of those flagged, roughly 42% truly were delinquent
F1-score	0.4482	Harmonic mean of precision and recall
Accuracy	0.9199	Below the 93.3% of always answering “no”

Table 4.2: Confusion matrix on the test set

	Predicted: not delinquent	Predicted: delinquent
Actual: not delinquent	26,621 (TN)	1,374 (FP)
Actual: delinquent	1,029 (FN)	976 (TP)

Read directly: of 2,005 borrowers who genuinely became seriously delinquent the model identified 976 and missed 1,029; of 27,995 who did not, it wrongly flagged 1,374.

Table 4.3: Per-class classification report

Class	Precision	Recall	F1-score	Support
0 — No serious delinquency	0.9628	0.9509	0.9568	27,995
1 — Serious delinquency	0.4153	0.4868	0.4482	2,005
Macro average	**0.6891**	**0.7189**	**0.7025**	**30,000**
Weighted average	0.9262	0.9199	0.9228	30,000

This table is the clearest demonstration of the imbalance problem. The model performs excellently on the majority class, with an F1 of 0.9568, and considerably less well on the minority class at 0.4482. The weighted average of 0.9228 looks impressive but is dominated by the 27,995 negative records and conveys almost nothing about the class the system exists to find. Where a single summary figure is required, the macro average of 0.7025 or the positive-class F1 should be quoted, never the weighted average.

Why the errors occur.

The application presents a chart that explains the recall ceiling directly. Each curve is one actual outcome group, drawn as a share of its own total because the negatives outnumber the positives by roughly fourteen to one.

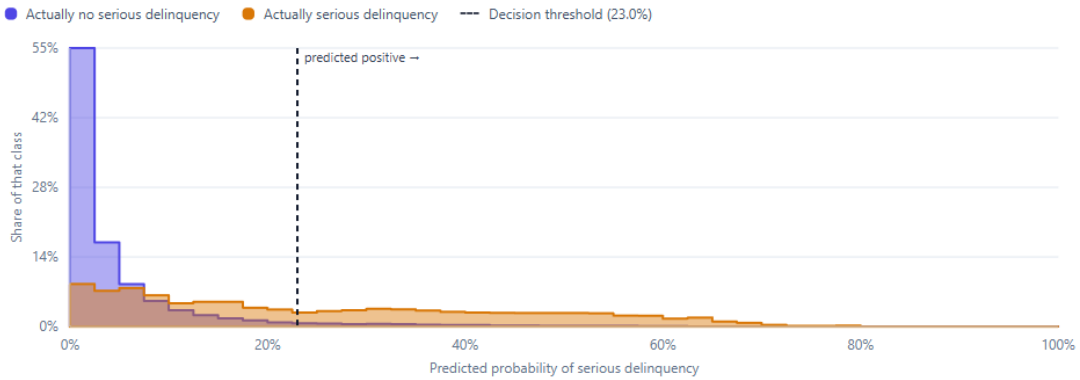


Figure 4.5: Predicted score distribution by actual outcome

Most borrowers who stayed current score near zero, which is why the model is accurate overall. Borrowers who did become delinquent are spread across the whole range rather than clustered at the high end, so a large share of them fall inside the same region as the healthy borrowers. That overlap, rather than any defect in the implementation, is what limits recall. Moving the threshold leftward captures more of the delinquent group but sweeps up more sound borrowers with it.

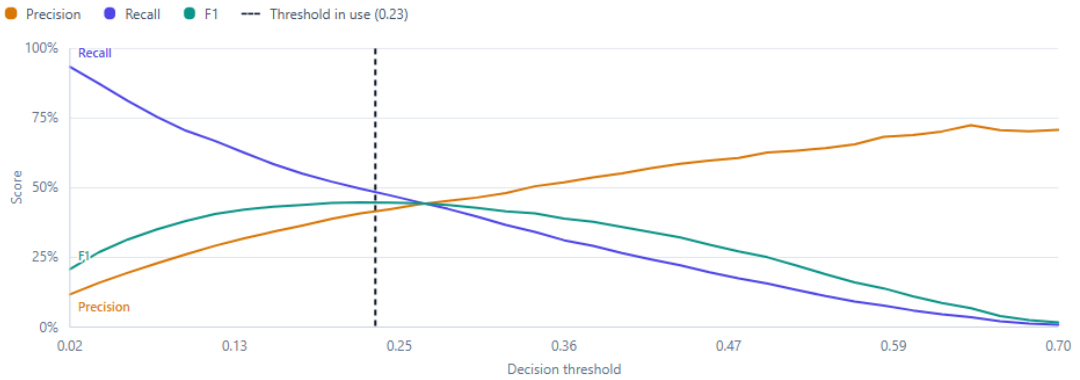


Figure 4.6: Precision, recall and F1 across decision thresholds

Table 4.4: Effect of the decision threshold on the test set

Threshold	Accuracy	Precision	Recall	F1	Flagged
0.1000	0.8469	0.2610	0.7047	0.3810	5,413
0.2000	0.9131	0.3884	0.5222	0.4454	2,696
0.2302	0.9199	0.4153	0.4868	0.4482	2,350
0.3000	0.9292	0.4650	0.3970	0.4283	1,712
0.5000	0.9374	0.6267	0.1566	0.2506	501

The same trained model produces very different behaviour depending on where the cut-off is placed. Lowering it to 0.10 raises recall above 0.70 but reduces precision to roughly 0.26, so three of every four flagged borrowers would be false alarms. Raising it to 0.50 gives precision above 0.62 but identifies only about one delinquency in six. Accuracy rises monotonically as the threshold rises, purely because the model

predicts the majority class more often, which is a further demonstration of why it is a poor guide here.

Stability of the result.

A single test score comes from one particular partition of the data. Five-fold cross-validation on the training and validation data, with the test split excluded throughout, gives a mean ROC-AUC of 0.8632 with a standard deviation of 0.0053, and a mean PR-AUC of 0.3978 ± 0.0148. The test values of 0.8654 and 0.4014 both fall within one standard deviation of their cross-validated means, so the reported performance is a property of the data and configuration rather than an artefact of one fortunate split.

Comparing in-sample with held-out performance also quantifies how much of the apparent skill is memorisation. ROC-AUC falls from 0.9329 on the fitted data to 0.8654 on held-out data, a gap of 0.0675, and PR-AUC falls by 0.1218. Some overfitting therefore remains despite the leaf-size regularisation, and this is reported rather than concealed.

CHAPTER 5: ADVANTAGES AND LIMITATIONS

5.1 Advantages

The Random Forest offered several concrete advantages for this problem. It captured non-linear relationships and interactions between payment history and credit usage without any manual feature engineering. Because trees split on rank order, it was unaffected by the extreme values present in the ratio attributes, which would have distorted a linear model and which were therefore left in place rather than capped. It required no feature scaling or categorical encoding.

Operationally it produces probability estimates rather than only hard labels, which is what a risk score requires, and the decision threshold can be moved after training without refitting. It also reports which attributes it relies on, which keeps the model inspectable. Training on 150,000 records completes in about a minute on a laptop, making the whole experiment reproducible.

5.2 Limitations

The most significant limitation is the data rather than the algorithm. The records describe United States consumers around 2011 and carry no date column, so no out-of-time validation is possible and present-day performance is unknown. The sampling frame is undocumented, so there is no basis for establishing which population the sample represents.

The predicted probabilities are not calibrated, and calibration was not evaluated. A predicted 0.30 should be read as a ranking score rather than as a claim that thirty in a hundred such borrowers default. Random Forest probabilities are typically compressed away from the extremes.

Fairness cannot be assessed, because the dataset contains no race, sex or geographic attributes. That absence is not protection: age is present and is used by the model, and attributes such as income and credit utilisation can act as proxies for protected characteristics. The absence of a measured bias is the absence of measurement, not evidence of fairness.

Finally, the chosen threshold maximises F1, which weights a missed delinquency and a false alarm equally. Real lending economics do not: the loss on a defaulted loan

generally exceeds the margin forgone on a declined application. Choosing the threshold properly would require loss-given-default and margin figures that the dataset does not contain.

5.3 Challenges

Three practical difficulties were encountered during implementation. The first concerned undocumented placeholder values: the three late-payment counters use the codes 96 and 98 in 269 records, and the data dictionary does not explain them. Those records are 54.65% positive against a base rate of 6.68%, so simply imputing them to the median would have discarded real signal while asserting that these borrowers had clean payment histories. The solution adopted was to impute the value but retain a missing-value indicator, which lets the model learn that the figure was unavailable without anyone having to guess what the codes mean.

The second challenge was deployment. The scientific Python stack together with the 70 MB model file produces a bundle of roughly 315 MB, which exceeds the limits of many hosting platforms. Separating the runtime dependencies from the training and plotting tools was necessary to fit within the available 500 MB allowance.

The third was avoiding train-serve skew. Any difference between the preprocessing applied during training and during inference would corrupt predictions silently. Combining the preprocessing and the classifier into a single saved pipeline removes the possibility entirely.

Table 5.1: Advantages and limitations compared

Advantages	Limitations
Captures non-linear interactions without manual feature engineering	Data describes 2011 US consumers; present-day performance unknown
Robust to extreme values; no scaling or encoding required	No date column, so out-of-time validation is impossible
Produces probability estimates, not only labels	Probabilities are uncalibrated and calibration was not evaluated
Threshold adjustable after training without refitting	Threshold optimises F1, which weights both error types equally
Reports feature importances, keeping the model inspectable	Importance describes splits, not causation, and is biased toward high-cardinality features
Trains in about a minute; fully reproducible with a fixed seed	No protected attributes present, so fairness cannot be tested

CHAPTER 6: FUTURE SCOPE

Several specific improvements follow directly from the limitations identified in the previous chapter.

Probability calibration. Fitting a calibration model on held-out data and reporting a reliability diagram and Brier score would establish whether the predicted probabilities can be read as real-world frequencies. This is the single change that would most increase the practical usefulness of the output.

Cost-sensitive threshold selection. Replacing the F1 criterion with an explicit cost matrix, once loss-given-default and margin figures are available, would place the operating point where it minimises expected loss rather than where it balances two error types that are not in fact equally costly.

Out-of-time validation. Obtaining a dataset with origination dates would permit training on an earlier period and testing on a later one, which is the standard validation approach for credit models precisely because random splits flatter them.

Permutation importance and per-prediction explanation. Permutation importance computed on held-out data is less biased than the impurity-based measure used here. Per-prediction attributions, for example through SHAP values, would support the adverse-action explanations that regulated lending requires.

Fairness assessment. With a dataset containing the relevant attributes, disparate-impact testing across protected groups could be performed, and mitigation applied where required.

More broadly, the direction of the field is toward models that are not merely accurate but auditable. Regulatory frameworks increasingly require that an automated credit decision can be explained to the person it affects, which favours methods whose reasoning can be inspected and argues against treating predictive performance as the only objective.

CHAPTER 7: CONCLUSION

This seminar studied the application of the Random Forest algorithm to the prediction of serious borrower delinquency, and implemented a complete working system to demonstrate it. A classifier was trained on 150,000 labelled records from the public Kaggle “Give Me Some Credit” dataset and evaluated on a held-out split of 30,000 records scored exactly once, achieving a ROC-AUC of 0.8654 and a PR-AUC of 0.4014 against a no-skill baseline of 0.0668.

The methodological findings are at least as important as the figures. Accuracy was shown to be an actively misleading metric for this problem, since a model that ignores its inputs scores higher than the trained one. Class weighting, the conventional remedy for imbalance, was measured and found to move the model along the precision-recall curve without lifting it, so an explicit decision threshold selected on validation data was used instead. Undocumented placeholder values in the dataset were found to carry substantial signal and were handled with missing-value indicators rather than imputed away. The residual gap between in-sample and held-out performance was measured and reported rather than omitted.

The practical component established that the path from a raw public dataset to a usable application is short when the engineering discipline is right: preprocessing held inside the saved model so that training and inference cannot diverge, the decision threshold stored alongside the model so the served cut-off and the reported metrics cannot disagree, and every documented figure generated from the model's own saved output rather than transcribed.

Table 7.1: Technology stack

Layer	Technology
Dataset	Kaggle “Give Me Some Credit” — 150,000 labelled records
Model	scikit-learn RandomForestClassifier, persisted with joblib
Data processing	Python, pandas, NumPy
Visualisation	matplotlib, seaborn; inline SVG in the web interface
Backend API	FastAPI, Pydantic, Uvicorn
Frontend	React, TypeScript, Vite, Tailwind CSS
Testing	pytest — 61 automated tests
Deployment	Vercel (frontend and backend as separate projects)

My own understanding of the significance of this topic is that the value of machine learning in credit risk lies less in squeezing out additional predictive accuracy than in making the basis of a decision explicit and consistent. The honest conclusion of this work is that the system built here extracts real and measurable signal from borrower data, and is simultaneously nowhere near fit to decide anybody's credit application. Holding both of those statements at once is, I think, the central skill the exercise teaches.

REFERENCES / BIBLIOGRAPHY

[1] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] Kaggle and Credit Fusion, “Give Me Some Credit.” Available: https://www.kaggle.com/competitions/GiveMeSomeCredit/data. Accessed: October 24, 2026.

[4] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” PLOS ONE, vol. 10, no. 3, 2015.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[6] T. Hastie, R. Tibshirani and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. Springer, 2009.

[7] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 3rd ed. O’Reilly Media, 2022.

[8] P. Bhalerao, “Loan Default Risk Prediction — project source code.” Available: https://github.com/prasadbhalerao1/ML_Project. Accessed: October 24, 2026.

[9] P. Bhalerao, “Loan Delinquency Risk — deployed application.” Available: https://delinquency-risk.vercel.app. Accessed: October 24, 2026.